



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5
по курсу «Разработка параллельных и
распределенных программ»
Синхронизация потоков

Студент: Федукон А. А.

Группа: ИУ9-52Б

Преподаватель: Царев А. С.

Москва, 2025 г.

Содержание

Постановка задачи	2
Практическая реализация	3
Код программы	3
Вывод программы	9
Вывод	11
Характеристики компьютера	11

Постановка задачи

1. Использование барьерной синхронизации — задача «эволюция».

Реализовать параллельную модель клеточного автомата на прямоугольной матрице размеров $M \times K$:

- Каждая клетка содержит 0 или 1; начальная матрица заполняется случайно.
- Правила эволюции стандартные (см. условие): у живой клетки меньше 2 соседей — смерть, 2 или 3 — выживает, больше 3 — смерть; пустая клетка с ровно 3 соседями становится живой.
- Граничные клетки имеют тороидальную геометрию (соседи с противоположной стороны).
- Запустить заданное число шагов T моделирования при помощи p потоков. Каждый поток отвечает за непрерывную горизонтальную (или вертикальную) полосу матрицы. Результаты шага вычисляются в отдельной «новой» матрице.
- После завершения вычисления шага все потоки синхронизировать барьером; затем содержимое новой матрицы копируется в исходную (копирование может выполняться потоками параллельно при соблюдении барьера).

- Поскольку поток видит только свою полосу, при обращении к элементам, лежащим за границей своей полосы, он должен получить их у соседних потоков (реализовать обмен граничными полосами через каналы/общую структуру с синхронизацией или читать соседние границы перед началом шага).
- Измерить среднее время одного шага при параллельной реализации (различные n) и сравнить со средней длительностью шага однопоточной реализации на том же входе. В отчёте привести методику измерения, таблицу результатов и краткий вывод.

2. Использование синхронизаций чтения и записи (reader/writer).

Написать программу, в которой n потоков генерируют по R псевдослучайных чисел в диапазоне $[0, 1000]$ и пытаются помещать их в конец однонаправленного связного списка только если такого числа там ещё нет:

- Поиск наличия числа в списке выполнять под общей блокировкой чтения (shared/read lock).
- Если число не найдено, перейти к блокировке записи (exclusive/write lock), и перед вставкой повторно проверить отсутствие числа (возможна «гонка» между чтением и захватом записи).
- Вставлять новый элемент в конец списка только под записью.
- По завершении всех потоков основной поток должен пройти по списку и проверить отсутствие повторяющихся значений.
- В отчёте привести исходный код программы и короткий отчёт о проверке корректности (например, длина списка и отсутствие дубликатов).

Практическая реализация

Код программы

Листинг 1: Исходный код программы для задачи 1

```
1 use rand::rngs::StdRng;
2 use rand::{Rng, SeedableRng};
3 use std::sync::{
4     Arc, Barrier,
5     atomic::{AtomicU8, Ordering},
6 };
7 use std::thread;
8 use std::time::Instant;
9
10 const ROWS: usize = 4096;
11 const COLS: usize = 4096;
12 const STEPS: usize = 1; // количество шагов эволюции
13 const THREADS: usize = 16;
14 const SEED: u64 = 1; // сид для генератора случайных чисел
15
16 #[inline(always)]
17 fn evolve(alive: u8, n: u32) -> u8 {
18     if alive == 1 {
19         if n < 2 {
20             0
21         } else if n == 2 || n == 3 {
22             1
23         } else {
24             0
25         }
26     } else {
27         if n == 3 { 1 } else { 0 }
28     }
29 }
30
31 struct Inboxes {
32     top: Vec<AtomicU8>, // сосед сверху пишет сюда
33     bottom: Vec<AtomicU8>, // сосед снизу пишет сюда
34 }
35
36 fn make_inboxes() -> Inboxes {
37     // создаем буферы для верхней и нижней граничных строк потока
38     Inboxes {
39         top: (0..COLS).map(|_| AtomicU8::new(0)).collect(),
40         bottom: (0..COLS).map(|_| AtomicU8::new(0)).collect(),
41     }
42 }
43
44 fn main() {
```

```

45     let threads_n = THREADS.min(ROWS).max(1); // проверяем что потоков н
         е больше строк
46
47     // Сколько строк на поток
48     let base = ROWS / threads_n;
49     // Сколько строк остаётся (прибавим к последнему потоку)
50     let rem = ROWS % threads_n;
51
52     let barrier = Arc::new(Barrier::new(threads_n));
53
54     // Для каждого потока создаем буферы для обмена граничными (по верти
         кали) строками
55     let inboxes: Vec<Arc<Inboxes>> = (0..threads_n).map(|_| Arc::new(
         make_inboxes())) .collect();
56
57     let start = Instant::now();
58
59     let mut thread_handles = Vec::with_capacity(threads_n);
60
61     for tid in 0..threads_n {
62         // строки для текущего потока (для последнего потока + остаток)
63         let my_rows = if tid == threads_n - 1 {
64             base + rem
65         } else {
66             base
67         };
68
69         // номера потоков-соседей (замыкаем по кругу)
70         let up = (tid + threads_n - 1) % threads_n;
71         let down = (tid + 1) % threads_n;
72
73         // создаем ссылки на
74         let b = barrier.clone();
75         let me = inboxes[tid].clone();
76         let up_in = inboxes[up].clone();
77         let down_in = inboxes[down].clone();
78
79         thread_handles.push(thread::spawn(move || -> u64 {
80             // Старые строки матрицы
81             let mut local = vec![0u8; my_rows * COLS];
82             // Новые строки матрицы
83             let mut next = vec![0u8; my_rows * COLS];
84
85             // скармливаем сид + tid для получения разных значений на ка
         ждом потоке
86             let mut rng =

```

```

87         StdRng::seed_from_u64(SEED ^ (tid as u64).wrapping_mul(0
x9E3779B97F4A7C15));
88         // заполняем полосу случайными 0 и 1
89         for x in &mut local {
90             *x = if rng.random_bool(0.5) { 1 } else { 0 };
91         }
92
93         for _ in 0..STEPS {
94             // ----- 1: обновляем границы
95             // отправляем верхнюю и нижнюю строки соседям
96             let last_off = (my_rows - 1) * COLS;
97             for j in 0..COLS {
98                 // верхняя строка -> up.bottom
99                 up_in.bottom[j].store(local[j], Ordering::Release);
100                 // нижняя строка -> down.top
101                 down_in.top[j].store(local[last_off + j], Ordering::
Release);
102             }
103             // Ждем пока все потоки отправят свои граничные строки
104             b.wait();
105
106             // ----- 2: вычисляем следующее состояние для всех стро
к этого потока
107             for i in 0..my_rows {
108                 // смещение для текущей строки
109                 let row_off = i * COLS;
110                 // флаги. Является ли верхняя/нижняя строка гранично
й?
111                 let up_is_inbox = i == 0;
112                 let dn_is_inbox = i + 1 == my_rows;
113
114                 // проходим по всем столбцам
115                 for j in 0..COLS {
116                     let jl = if j == 0 { COLS - 1 } else { j - 1 };
117                     let jr = if j + 1 == COLS { 0 } else { j + 1 };
118
119                     let mut n: u32 = 0;
120
121                     // верхняя часть
122                     if up_is_inbox {
123                         // берем атомики из буфера
124                         n += me.top[jl].load(Ordering::Acquire) as
u32;
125
126                         n += me.top[j].load(Ordering::Acquire) as
u32;

```

```

126         n += me.top[jr].load(Ordering::Acquire) as
u32;
127     } else {
128         let up_off = (i - 1) * COLS;
129         n += local[up_off + jl] as u32;
130         n += local[up_off + j] as u32;
131         n += local[up_off + jr] as u32;
132     }
133
134     // лево и право
135     n += local[row_off + jl] as u32;
136     n += local[row_off + jr] as u32;
137
138     // низ
139     if dn_is_inbox {
140         // берем атомики из буфера
141         n += me.bottom[jl].load(Ordering::Acquire)
as u32;
142         n += me.bottom[j].load(Ordering::Acquire) as
u32;
143         n += me.bottom[jr].load(Ordering::Acquire)
as u32;
144     } else {
145         let dn_off = (i + 1) * COLS;
146         n += local[dn_off + jl] as u32;
147         n += local[dn_off + j] as u32;
148         n += local[dn_off + jr] as u32;
149     }
150
151     // Вызываем эволюцию и записываем в next
152     let alive = local[row_off + j];
153     next[row_off + j] = evolve(alive, n);
154 }
155 }
156 // Ждем пока все потоки подсчитают новое состояние
157 b.wait();
158
159 // ----- 3: меняем старое и новое состояние местами
160 std::mem::swap(&mut local, &mut next);
161 // Ждем пока все потоки закончат обновление состояния
162 b.wait();
163 }
164
165 // Вычисляем контрольную сумму по локальному куску
166 local.iter().map(|&x| x as u64).sum()
167 }));

```

```

168     }
169
170     // Собираем контрольные суммы от всех потоков
171     let mut checksum = 0u64;
172     for h in thread_handles {
173         checksum = checksum.wrapping_add(h.join().unwrap());
174     }
175
176     println!("{}", ms (checksum={}), start.elapsed().as_millis(),
checksum);
177 }

```

Листинг 2: Исходный код программы для задачи 2

```

1     use parking_lot::RwLock;
2 use rand::{rngs::ThreadRng, Rng};
3 use std::collections::{HashSet, LinkedList};
4 use std::sync::Arc;
5 use std::thread;
6 use std::time::Instant;
7
8 const THREADS: usize = 16;
9 const COUNT: usize = 1_000_000;
10
11 fn contains(list: &LinkedList<i32>, v: i32) -> bool {
12     list.iter().any(|&x| x == v)
13 }
14
15 fn worker(num_values: usize, list: Arc<RwLock<LinkedList<i32>>>) {
16     let mut rng: ThreadRng = rand::thread_rng();
17
18     for _ in 0..num_values {
19         let v: i32 = rng.gen_range(0..=1000);
20
21         // 1) Проверка под read-lock
22         {
23             let guard = list.read();
24             if contains(&guard, v) {
25                 continue;
26             }
27         }
28
29         // 2) Запись под write-lock + повторная проверка
30         {
31             let mut guard = list.write();
32             if !contains(&guard, v) {
33                 guard.push_back(v);

```



```

34         }
35     }
36 }
37 }
38
39 fn main() {
40     let start_time = Instant::now();
41     let list = Arc::new(RwLock::new(LinkedList::<i32>::new()));
42
43     let mut thread_handles = Vec::with_capacity(THREADS);
44     for _ in 0..THREADS {
45         let l = Arc::clone(&list);
46         thread_handles.push(thread::spawn(move || worker(COUNT, l)));
47     }
48
49     // Ждем пока все потоки завершатся
50     for h in thread_handles {
51         h.join().unwrap();
52     }
53
54     // Вывод результата
55     let values: Vec<i32> = {
56         let guard = list.read();
57         guard.iter().copied().collect()
58     };
59
60     // print!("Result list: ");
61     // for v in &values {
62         // print!("{v} ");
63     // }
64     // println!();
65     let elapsed = start_time.elapsed().as_millis();
66     println!("Elapsed time: {} ms", elapsed);
67
68     // Проверка на дубли
69     let mut seen = HashSet::new();
70     for &v in &values {
71         if !seen.insert(v) {
72             panic!("Duplicate value found: {v}");
73         }
74     }
75     println!("No duplicates found. Total unique values: {}", seen.len());
76 }

```

Вывод программы

1. Вывод программы для задачи 1.

В листинге 3 приведены результаты замеров времени выполнения программы для различных количеств потоков и двух значений количества шагов эволюции ($STEPS = 10$ и $STEPS = 1$). Также приведены контрольные суммы (checksum) для проверки корректности работы программы.

2. Вывод программы для задачи 2.

В листинге 4 приведены результаты замеров времени выполнения программы для различных количеств потоков. Программа успешно завершилась без обнаружения дубликатов в связном списке.

Листинг 3: Вывод программы 1

```
1 const ROWS: usize = 4096; // число строк матрицы
2 const COLS: usize = 4096; // число столбцов матрицы
3 const THREADS: usize = 2; // количество потоков
4
5 const STEPS: usize = 10; // количество шагов эволюции
6
7 1 : 25028 ms (checksum=3359996)
8 2 : 12691 ms (checksum=3358243)
9 4 : 6197 ms (checksum=3362872)
10 8 : 4124 ms (checksum=3360288)
11 16 : 3599 ms (checksum=3359891)
12
13 const STEPS: usize = 1; // количество шагов эволюции
14
15 1 : 9596 ms (checksum=4593682)
16 2 : 4540 ms (checksum=4595231)
17 4 : 2434 ms (checksum=4592151)
18 8 : 1350 ms (checksum=4588302)
19 16 : 1279 ms (checksum=4589131)
```

Листинг 4: Вывод программы 2

```
1 const COUNT: usize = 1_000_000;
2
3 1 : Elapsed time: 3357 ms
4 2 : Elapsed time: 3486 ms
5 4 : Elapsed time: 3595 ms
6 8 : Elapsed time: 5621 ms
7 16 : Elapsed time: 9891 ms
```

Вывод

В ходе выполнения лабораторной работы были реализованы две задачи, требующие использования различных механизмов синхронизации потоков. Первая задача, связанная с барьерной синхронизацией, продемонстрировала эффективное распределение вычислений между потоками при моделировании клеточного автомата. Вторая задача, использующая синхронизацию чтения и записи, показала важность правильного управления доступом к общим данным для предотвращения гонок и обеспечения целостности данных. В первом случае полученные результаты замеров времени выполнения программ подтвердили эффективность выбранных подходов к синхронизации в многопоточных приложениях. Однако во втором случае наблюдалось увеличение времени выполнения с ростом количества потоков, что связано с накладными расходами на управление блокировками при использовании RWLock. В целом, работа позволила углубить понимание механизмов синхронизации в параллельном программировании и их влияния на производительность приложений.

Характеристики компьютера

Листинг 5: Характеристики компьютера

```
1 > inxi --cpu --memory
2 Memory:
3   System RAM: total: 16 GiB available: 14.8 GiB used: 10.48 GiB (70.8%)
4   Array-1: capacity: 64 GiB slots: 4 modules: 4 EC: None
5   Device-1: Channel-A DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
6   Device-2: Channel-B DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
7   Device-3: Channel-C DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
8   Device-4: Channel-D DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
9 CPU:
10  Info: 6-core model: AMD Ryzen 5 7640HS w/ Radeon 760M Graphics bits:
    64
11    type: MT MCP cache: L2: 6 MiB
12  Speed (MHz): avg: 1200 min/max: 400/5017 cores: 1: 1390 2: 1100 3:
    1100
13    4: 1100 5: 1100 6: 1100 7: 1396 8: 1100 9: 1432 10: 1100 11: 1389
    12: 1100
```